

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII ȘTIINȚIFICE



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

FPWAM - O implementare a mașinii abstracte Warren pe FPGA

LUCRARE DE LICENȚĂ

Absolvent: **Bogdan Daniel ARDELEAN**
Conducător științific: **Dr. Ing. Zoltan Francisc BARUCH**

2017



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE**

DECAN,
Prof. dr. ing. Liviu MICLEA

DIRECTOR DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Absolvent: **Bogdan Daniel ARDELEAN**

FPWAM - O implementare a mașinii abstracte Warren pe FPGA

1. **Enunțul temei:** *Scurtă descriere a temei lucrării de licență și datele inițiale*
2. **Conținutul lucrării:** *(enumerarea părților componente) Exemplu: Pagina de prezentare, aprecierile coordonatorului de lucrare, titlul capitolului 1, titlul capitolului 2, titlul capitolului n, bibliografie, anexe.*
3. **Locul documentării:** *Exemplu: Universitatea Tehnică din Cluj-Napoca, Departamentul Calculatoare*
4. **Consultanți:**
5. **Data emiterii temei:** 1 Noiembrie 2015
6. **Date predării:** 17 Februarie 2017 *(se va completa data predării)*

Absolvent: _____

Coordonator științific: _____



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE**

**Declarație pe proprie răspundere privind
autenticitatea lucrării de licență**

Subsemnatul(a)

_____, legitimat(ă)
cu _____ seria _____ nr. _____
CNP _____, autorul lucrării _____

elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Specializarea _____ din cadrul Universității Tehnice din Cluj-Napoca, sesiunea _____ a anului universitar _____, declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de licență*.

Data

Nume, Prenume

Semnătura

De citit înainte (această pagină se va elimina din versiunea finală):

1. Cele trei pagini anterioare (foaie de capăt, foaie sumar, declarație) se vor lista pe foi separate (nu față-verso), fiind incluse în lucrarea listată. Foaia de sumar (a doua) necesită semnătura absolventului, respectiv a coordonatorului. Pe declarație se trece data când se predă lucrarea la secretarii de comisie.
2. Pe foaia de capăt, se va trece corect titulatura cadrului didactic îndrumător, în engleză (consultați pagina de unde ați descărcat acest document pentru lista cadrelor didactice cu titulaturile lor).
3. Documentul curent **nu** a fost creat în MS Office. E posibil să fie mici diferențe de formatare.
4. Cuprinsul începe pe pagina nouă, impară (dacă se face listare față-verso), prima pagină din capitolul Introducere tot așa, fiind numerotată cu 1.
5. Vizualizați (recomandabil și în timpul editării) acest document
6. Fiecare capitol începe pe pagină nouă.
7. Folosiți stilurile predefinite (Headings, Figure, Table, Normal, etc.)
8. Marginile la pagini nu se modifică.
9. Respectați restul instrucțiunilor din fiecare capitol.

Cuprins

Capitolul 1	Introducere - Contextul proiectului	11
1.1	Contextul proiectului	11
1.1.1	FPGA	11
1.1.2	Prolog	12
1.1.3	Nu m-am decis	12
Capitolul 2	Obiectivele Proiectului	13
2.1	Scop	13
2.2	Obiective generale	13
2.3	Obiective S.M.A.R.T.	14
Capitolul 3	Studiu Bibliografic	17
3.1	Sisteme de calcul reconfigurabile și FPGA	17
3.1.1	Circuitele integrate FPGA	17
3.2	Prolog	18
3.2.1	Limbajul Prolog	19
3.2.2	Arhitectura WAM	21
3.3	Abordări similare	21
Capitolul 4	Analiză și Fundamentare Teoretică	22
4.1	Hardware - Arhitectura WAM și soluția propusă	22
4.1.1	Arhitectura memoriei	22
4.1.2	Registrii	23
4.1.3	Reprezentarea termenilor	24
4.1.4	Stiva-AND și Stiva-OR	25
4.1.5	Apelul de proceduri	28
4.1.6	Indexarea codului	28
4.1.7	Setul abstract de instrucțiuni	29
4.1.8	Instrucțiuni de tipul PUT	29
4.1.9	Operațiile auxiliare	36
4.2	Soluția programelor software	36
4.2.1	Compilarea PROLOG-WAM	36

4.2.2	Translatarea din WAM în cod mașină FPWAM	37
4.2.3	Mediul de executare	37
Capitolul 5	Proiectare de Detaliu și Implementare	38
5.1	Arhitectura generală a sistemului	38
5.2	Modulul hardware	39
5.2.1	Pachetul FpwamPkg.vhd	39
5.2.2	Entitatea TopLevel.vhd și arhitectura generală	42
5.2.3	Entitatea DataFlowControl.vhd sau DFC	42
5.2.4	Entitatea GPR.vhd sau General Purpose Registers	44
5.2.5	Entitatea Memory.vhd	44
5.2.6	Operațiile auxiliare	46
5.3	Modulul software	46
5.3.1	Arhitectura conceptuală	46
5.3.2	C API-ul expus al translatorului	47
5.3.3	Blocul C și parser-ul de fișiere WAM	48
5.3.4	wam2FPWAM.h,c	48
Capitolul 6	Testare și Validare	50
6.1	Titlu	50
6.2	Alt titlu	50
Capitolul 7	Manual de Instalare și Utilizare	51
7.1	Titlu	51
7.2	Alt titlu	51
Capitolul 8	Concluzii	52
8.1	Titlu	52
8.2	Alt titlu	52
Bibliografie		53
Anexa A	Secțiuni relevante din cod	54
Anexa B	Alte informații relevante (demonstrații etc.)	55
Anexa C	Lucrări publicate (dacă există)	56

Capitolul 1

Introducere - Contextul proiectului

1.1 Contextul proiectului

Încă din secolul XVII, cu mult înaintea calculatoarelor electronice, exista termenul *calculator*. Până la jumătatea secolului XX, denotația lui era cu totul alta față de cea pe care o știm noi astăzi. Sensul acestui cuvânt referindu-se în principal la o meserie. Rolul asociat unei persoane care practica meseria de calculator, era de a efectua calcule matematice complexe în funcție de necesitățile clientului. Odată cu trecerea timpului, știința a avut întâmpinat provocări la care persoanele calculator nu mai puteau ține pasul. Această lipsă a puterii de calcul și inventarea tranzistorului au concentrat mediul industrial și academic în dezvoltarea sistemelor pe care astăzi le folosim zilnic.

1.1.1 FPGA

În momentul de față, majoritatea dispozitivelor de calcul existente au la bază un procesor cu scop general care execută un program croit după necesitățile utilizatorului. În ciuda îmbunătățirilor constante și a flexibilității în dezvoltarea programelor acestor procesoare, arhitectura lor fixă nu se mulează întotdeauna suficient de mult pe problema pe care încearcă să o rezolve, rezultatul fiind nesatisfăcător din punctul de vedere a timpului de execuție și a amprentei energetice.

La jumătatea anilor 1980 a apărut primul circuit integrat comercial de tipul FPGA(Field-Programmable Gate Array)-referință. După cum le sugerează și numele, ele sunt niște circuite integrate care pot fi configurate de câte ori este nevoie de către utilizator. Avantajul acestor circuite este în principal natura lor paralelă și consumul redus de energie electrică. Datorită avansărilor tehnologice și prețului din ce în ce mai redus, ele au crescut în popularitate și au ajuns să fie folosite în multe domenii precum: *Calcul de înaltă performanță, Auto, Audio, Medical, etc.* - referință

1.1.2 Prolog

Prolog a fost inventat la începutul anilor 1970 de către Alain Colmerauer și colegii lui de la Universitatea din Marseille-ref. El este printre primele limbaje de uz general care au ca paradigmă de baza programarea logică. Prima implementare a acestui limbaj a fost sub forma unui interpretor scris în FORTRAN chiar de către grupul din care Colmerauer făcea parte. Chiar și sub această formă rudimentară realizările au fost mari deoarece s-a demonstrat viabilitatea unui limbaj de programare logic. După aproximativ un deceniu, David H. D. Warren a rafinat și abstractizat principiile de implementare a limbajului sub forma unei mașini abstracte care este cunoscută astăzi sub numele de WAM. Majoritatea implementărilor de success ale limbajului Prolog folosesc ca referință de bază mașina abstractă mai sus amintită datorită simplității, eficienței și robusteții care au trecut testul timpului. Datorită popularității WAM, au fost scrise multe lucrări științifice care au adus îmbunătățiri arhitecturale a mașinii inițiale cât și multe extinderi. Una dintre cele mai sensibile subiecte este paralelizarea eficientă a acestei arhitecturi în speranța de a obține rezultate practice mai bune.

Prolog este de obicei asociat cu domeniul inteligenței artificiale și a lingvisticii computaționale. Cele mai des întâlnite aplicații ale limbajului fac parte din sfera mediului academic—e.g, sisteme expert, demonstrarea automată a teoremelor, NLP, planificare automată, etc.

1.1.3 Nu m-am decis

În contextul actual, atât domeniul circuitelor reconfigurabile, cât și domeniul inteligenței artificiale se dezvoltă cu un ritm accelerat, fiind folosite tot mai des în produse comerciale—e.g., aproape toate produsele Google folosesc elemente de inteligență artificială (Translate, Photos, etc.)-ref, Microsoft exploatează circuitele reconfigurabile pentru a accelera motorul de căutare BING-ref. În principiu orice aplicație care are la bază inteligența artificială are nevoie de un mediu de antrenare sau executare de înaltă performanță pe care procesoarele obișnuite nu îl pot oferi. Încercarea mea urmărește să utilizeze natura paralelă și performanța crescută a circuitelor FPGA, pentru a soluționa problema puterii de calcul în contextul aplicațiilor bazate pe limbajul Prolog.

Capitolul 2

Obiectivele Proiectului

2.1 Scop

Scopul acestui proiect este de a realiza un sistem dedicat de execuție a programelor scrise în limbajul Prolog, urmând a fi folosit de către cercetători și ingineri care activează în domeniul inteligenței artificiale și a programării logice. Acest sistem va fi compus dintr-un procesor special optimizat pentru executarea unui set de instrucțiuni ales din limbajul Prolog, implementat pe un circuit reprogramabil de tipul FPGA, având la bază arhitectura WAM propusă de către David H. D. Warren. El va fi însoțit de o suită de programe compatibile cu sistemele de operare din familia UNIX, care vor servi ca interfață dintre calculatorul gazdă și noul procesor, FPWAM (FPGA PROLOG WAM).

2.2 Obiective generale

Până la sfârșitul dezvoltării acestui proiect, se urmăresc atingerea următoarelor obiective principale:

- Studiul în detaliu a mașinii propuse de către David H. D. Warren este imperios necesară deoarece ea va sta la baza procesorului FPWAM.
- Identificarea posibilelor instrucțiuni care pot fi optimizate.
- După studierea tuturor surselor teoretice, următorul pas va fi acela de a concepe o arhitectură fizică care să exploateze natura paralelă a circuitelor FPGA în implementarea algoritmilor prezenți în mașina WAM.
- Arhitectura proiectată să fie scalabilă, astfel încât să permită adăugarea cu ușurință a mai multor procesoare Prolog.
- Implementarea efectivă pe un circuit integrat de tip FPGA.

- Creșterea frecvenței de ceas prin metode de optimizare a circuitului.
- Din cauza faptului că proiectul de față nu își propune să construiască un compilator de Prolog, este necesară alegerea unui existent cu care FPWAM să fie compatibil.
- Dezvoltarea unui translator din instrucțiuni WAM în cod mașină FPWAM.
- Dezvoltarea unei aplicații de comunicare între calculatorul și FPWAM.
- Suita de programe dezvoltate să fie compatibile cu sistemele de operare din familia UNIX.

2.3 Obiective S.M.A.R.T.

Principiile S.M.A.R.T.

De cele mai multe ori scopurile și obiectivele sunt formulate fără o structură anume, având consecințe destul de grave precum imposibilitatea de urmărire progresul și lipsa de predictibilitate. Metodologia S.M.A.R.T. de formulare a obiectivelor și a scopurilor este folosită cel mai des în management-ul proiectelor fiind una dintre cele mai eficiente metode în acest sens. Pentru ca un obiectiv să îndeplinească criteriile S.M.A.R.T. el trebuie să îndeplinească următoarele principii:

- Specific: Cu cât obiectivul este mai specific, cu atât șansele sunt mai mari să-l obținem. El trebuie să răspundă la întrebări precum ce vrem să obținem exact, unde, cum sau când.
- Measurable (măsurabil): Ca un obiectiv să fie măsurabil el trebuie să fie împărțit în elementele măsurabile, rezultatul fiind apreciat după dovezile concrete.
- Attainable (realizabil): Este foarte important să punem în balanță efortul, timpul și alte costuri cu profitul și alte priorități pe care le avem.
- Relevant: Obiectivul trebuie să contribuie la atingerea scopului dorit.
- Time-bound (limitat de timp): Cu toții știm că termenele limită pun oamenii în mișcare, așa că obiectivele trebuie să fie constrânse de timp.

Obiectivele S.M.A.R.T. ale proiectului

Luând în considerare principiile enumerate mai sus, s-au formulat următoarele obiective S.M.A.R.T. ale proiectului:

- Realizarea unui studiu teoretic asupra implementărilor limbajului Prolog:
 - S: Studiarea și înțelegerea articolului scris de către David H. D. Warren în care introduce pentru prima oară arhitectura WAM. Studiarea a altor alternative.
 - M: Documentarea din cel puțin două surse diferite și schițarea unei sinteze a acestora.

- A: Prolog fiind un limbaj de programare popular, se găsesc multe surse din care se pot extrage informații relevante.
- R: Studiind soluțiile existente și elaborând o sinteză, putem să punem în balanță avantajele și dezavantajele lor, ducând în final la alegerea optimă.
- T: Studiul nu trebuie să depășească 3 luni de zile.
- Implementarea procesorului M0 pe FPGA:
 - S: În versiunea M0 se urmărește ca procesorul să poată unifica un fapt cu o întrebare Prolog predefinite în memoria de program.
 - M: Pentru ca acest obiectiv să fie îndeplinit următoarele elemente trebuie să fie implementate: arhitectura de bază formată din memorie și regiștrii, algoritmul de unificare a faptului cu întrebarea, un set minim de instrucțiuni WAM care să permită unificarea
 - A: Fiind la începutul proiectului, există timp pentru experimente până la definirea arhitecturii.
 - R: Atingerea acestui obiectiv este un pas foarte important deoarece demonstrează viabilitatea unei implementări hardware a limbajului.
 - T: Cu tot cu implementare și experimente, acest proces nu trebuie să dureze mai mult de 1 lună.
- Dezvoltarea suitei de programe pentru mediul de executare a FPWAM:
 - S: Suita de programe trebuie să permită compilarea și executarea de programe Prolog pe FPWAM. Modulul de compilare va utiliza GPLC (Gnu Prolog Compiler).
 - M: Obținerea unor executabile testate și compatibile cu sistemele de operare din familia UNIX.
 - A: Compilarea Prolog->WAM este efectuată de către GPLC reducând considerabil complexitatea acestui obiectiv.
 - R: Datorită acestor programe, interacțiunea dintre FPWAM și utilizator va fi posibilă într-un mod natural și eficient.
 - T: Funcționalitățile de baza trebuie să fie implementate în maxim 3 zile.
- Implementarea procesorului M3 pe FPGA:
 - S: În versiunea M3 procesorul va putea să execute orice program Prolog care respectă subsetul selectat de instrucțiuni.
 - A: Pentru ca acest obiectiv să fie îndeplinit următoarele funcționalități trebuie să fie implementate: mecanismul de apel a unui predicat, mecanismul de backtracking, 90% din instrucțiunile WAM.

- R: Acest obiectiv duce proiectul aproape de îndeplinirea scopului final. În acest stadiu se pot face comparații și teste amănunțite.
- T: Acest obiectiv trebuie să fie îndeplinit la cel puțin 15 zile înaintea de predarea proiectului.

Capitolul 3

Studiu Bibliografic

3.1 Sisteme de calcul reconfigurabile și FPGA

În ultimul deceniu, sistemele de calcul reconfigurabile au devenit foarte populare datorită capacității lor de a îmbina flexibilitatea unui produs software cu performanța ridicată a puterii de procesare a componentelor hardware. Acest concept a fost introdus pentru prima oară de către Gerlad Estrin într-un articol publicat în anii 1960-ref. El propunea un sistem format dintr-un procesor standard și o matrice de primitive hardware care putea fi configurată în funcție de necesitatea aplicației. Din cauza limitărilor tehnologice de la acea vreme, acest domeniu nu prea a fost în atenția cercetătorilor. După mai bine de două decenii producătorul de circuite integrate Xilinx inventează și lansează pentru prima oară circuitele field-programmable gate array (FPGA). Datorită comercializarea acestor circuite și performanțele ridicate ale acestora, domeniul sistemelor reconfigurabile a trecut printr-o perioadă de renaștere în care s-au publicat numeroase articole demonstrând viabilitatea acestor sisteme-ref.

3.1.1 Circuitele integrate FPGA

Un FPGA este un circuit integrat destinat configurării de către utilizator sau unui arhitect după procesul de fabricație. Configurația lor este specificată similar cu cea a circuitelor ASIC (application-specific integrated circuit) utilizând un limbaj de descriere hardware (HDL). Cea mai comună arhitectură pe care o adoptă aceste circuite conține o matrice de blocuri logice configurabile (CLB), celule de intrare/ieșire și canale de rutare. O imagine explicativă se găsește în figura 3.1.

Elementele CLB

Elementele CLB sunt blocurile fundamentale din care sunt formate FPGA-urile. Ele sunt alcătuite din mai multe celule logice (slice-uri). O versiune simplificată a unui slice se găsește în figura 3.2. De regulă el conține:

- LUT(look-up table) cu 4 intrări

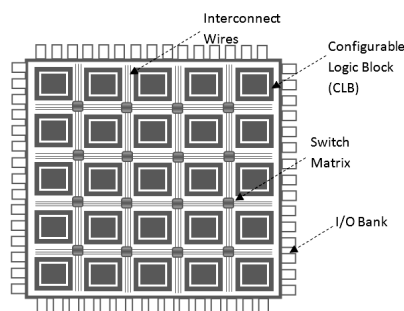


Figura 3.1: Arhitectura unui FPGA (<http://allaboutfpga.com/fpga-architecture/>)

- Sumator complet
- Bistabil de tip D
- Multiplexoare pentru selecția modului *normal sau aritmetic* în combinație cu modul de ieșire *sincron sau asincron*

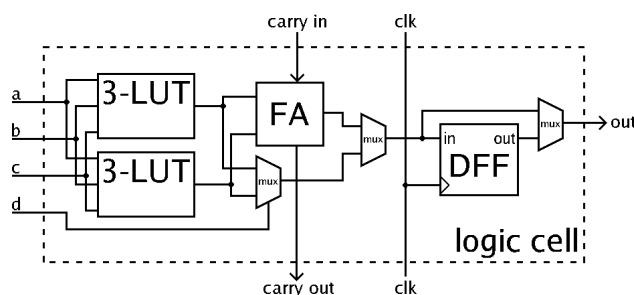


Figura 3.2: SLICE

Elemente de rutare

Elementele de rutare din FPGA sunt folosite pentru interconectarea blocurilor logice printr-o matrice de fire și comutatoare programabile. O schemă sumară se găsește în figura 3.3. În general rutarea între elementele programabile este nesegmentată, ceea ce înseamnă că fiecare segment rutat își găsește un capăt într-un bloc logic iar celălalt într-un comutator programabil. Pentru viteze ridicate, unele arhitecturi FPGA folosesc linii de rutare care trec prin mai multe elemente CLB.

3.2 Prolog

După cum am amintit și în 1 limbajul Prolog a fost inventat la începutul anilor 1970 de către Alain Colmerauer și colegii lui de la Universitatea din Marseille-ref. El este printre primele

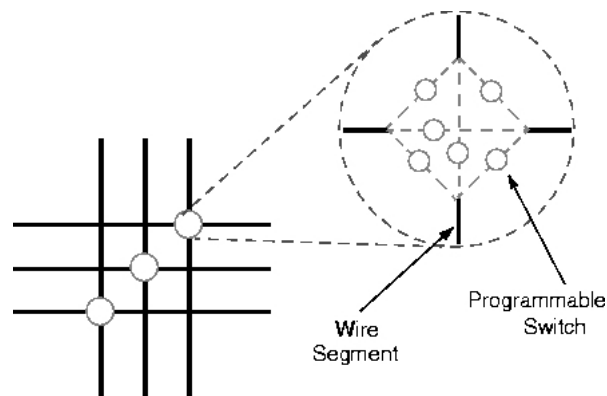


Figura 3.3: Matrice de rutare și comutator programabil

limbaje de uz general care au ca paradigmă de baza programarea logică. Domeniile lui de aplicabilitate sunt de obicei cele din sfera inteligenței artificiale, mai exact a sistemelor bazate pe cunoștințe, sisteme expert, etc-ref.

3.2.1 Limbajul Prolog

Într-un program Prolog pot fi indentificate următoarele construcții de bază:

1. fapte
2. întrebări
3. termeni
4. reguli

Fapte

În limbajul Prolog putem face afirmații utilizând faptele. Faptele sunt niște itemi ori o relație între itemi. De exemplu:

```
respira(om). %omul respiră
program_inicializat.
```

Întrebări

Îndată ce faptele au fost definite, Prolog ne permite să interogăm baza de fapte:


```
?-program_inicializat.
yes.
?-respira(om).
yes.
?-respira(carte).
false.
?-respira(X), % interogare cu termen de tip variabilă
X = om
```

Termeni

Termenii în Prolog se grupează în următoarele categorii:

- simpli
 - variabile
 - constante
 - * atomi
 - * numere
- structurați

Reguli

Prologul permite formularea de reguli de tipul: ”Un fapt *a* este adevărat, dacă faptele *b..z* sunt adevărate.” Regula Prolog este formată dintr-un *cap* și un *corp* despărțite de simbolul *:-*. Exemplu:

```
a(X):-b(X), c(X), ... ,z(X) %a este adevărat daca b, c ... sunt adevărate
```

Mecanisme

Spre deosebire de alte limbaje, Prolog are integrate două mecanisme ce-l face unic printre alte limbaje de programare.

- *Mecanismul de unificare a termenilor*. Doi termeni se unifică doar dacă ei sunt egali sau dacă conțin variabile care se pot instanția uniform cu termeni astfel încât cei doi termeni inițiali să fie egali.
- *Mecanismul de backtracking*. În cazul în care programul ajunge într-o stare în care capul sau corpul unei reguli nu poate fi unificat, mecanismul de backtracking intră în acțiune și reface starea procesului la ultimul punct de alegere și îl execută. Acest procedeu continuă până când unificarea are succes sau nu mai există puncte de alegere, moment în care întrebarea eșuează.

3.2.2 Arhitectura WAM

După mai bine de un deceniu de la inventarea limbajului Prolog, în articolul -ref publicat de către David H. D. Warren se descrie o mașină abstractă care este definită printr-un set de instrucțiuni și o arhitectură a memoriei, cunoscută astăzi ca WAM. De atunci, WAM a devenit un standard pentru implementarea limbajului Prolog. Din cauza faptului că Warren descrie WAM-ul foarte sumar, este dificil pentru un cititor neexperimentat cu limbajul Prolog să înțeleagă arhitectura din lucrarea originală-ref. În -ref Hassan A IT-KACI adresează această problemă publicând o carte sub forma unui tutorial care construiește gradual arhitectura, având în minte principiile după care Warren s-a ghidat. Arhitectura WAM este una bazată pe stivă, împărțind similarități cu limbajele imperative: instrucțiuni de tipul call/return, mediu local. Setul de instrucțiuni permite emularea sau traducerea într-un cod mașină nativ.

3.3 Abordări similare

În -ref se propune un sistem hardware implementat pe un circuit integrat de tip FPGA pentru programare logică inductivă. Acest sistem are la bază arhitectura WAM optimizată prin metode de grupare a instrucțiunilor și asignare speculativă. Spre deosebire de abordarea propusă în această lucrare, acest sistem folosește mai multe procesoare WAM interconectate care să lucreze împreună la executarea întrebării Prolog. În tabelul 1 din capitolul 5 se demonstrează viabilitatea chiar și unui sistem secvențial, având performanța comparativă cu a unui procesor Pentium III tactat la 450MHz. Dezavantajele acestei abordări sunt tactarea procesorului de Prolog la 35MHz-ref, folosirea memoriei RAM distribuite în locul memoriei BlockRAM, forțând ca multe segmente să nu fie ținute în memoria internă a FPGA-ului din cauza lipsei de spațiu. Accesul la segmentele care sunt ținute înafara circuitului vor scădea performanța considerabil.

O altă abordare de implementare hardware a arhitecturii WAM propusă în -ref de către Evan Tick în 1983 încearcă să determine performanța maximă pe care o poate atinge o mașină Prolog secvențială. Această abordare nu prea este relevantă, deoarece tehnologia a avansat mult de atunci, dar studiul arată că se pot s-au obținut îmbunătățiri de până la 18% folosind componente hardware standard.

Capitolul 4

Analiză și Fundamentare Teoretică

În acest capitol se va prezenta pe cât posibil bazele teoretice a procesorului care urmează să fie implementat. Arhitectura abstractă WAM este una complicată, având un set de instrucțiuni stufos care nu este intuitiv pentru cititorul neexperimentat cu limbajul Prolog. Pentru înțelegerea în detaliu se recomandă lecturarea cărții -ref.

În paralel se vor discuta și modificările survenite pentru adaptarea ei la arhitectura care urmează să fie implementată pe FPGA. O altă parte ce va fi tratată aici este protocolul pentru comunicarea cu calculatorul gazdă, principiile din spatele translatorului WAM-FPWAM și ideea aplicației ce oferă utilizatorului un mediu de executare.

Partea legată de arhitectura WAM nu este

4.1 Hardware - Arhitectura WAM și soluția propusă

4.1.1 Arhitectura memoriei

După cum am mai amintit în capitolele anterioare WAM este un procesor bazat pe stivă. În specificația originală, memoria este împărțită în următoarele zone (în ordine crescătoare a spațiului de adrese) :

1. *CODE* - zonă de memorie destinată exclusiv păstrarea codului pentru faptele și regulile Prolog.
2. *HEAP* - zonă de memorie destinată exclusiv păstrarea oricărui tip de termen Prolog.
3. *STACK* - zonă de memorie despărțită logic în două secțiuni: stivă-AND și stivă-OR. Pe stiva-AND se reține de obicei cadrul local care conține variabilele locale, următorul mediu local și numărul instrucțiunii de întoarcere. Pe stiva-OR se țin informații pentru mecanismul de backtracking. Ele sunt legate de punctul de alegere și informații pentru refacerea stării procesorului la executarea operației backtracking.
4. *TRAIL* - zonă de memorie destinată ținerii evidenței variabilelor unificate.

5. *PDL* - zonă de memorie locală destinată algoritmului de unificare a termenilor.

Din motive pe care se vor discuta în capitolele următoare s-a introdus o nouă zonă de memorie care nu este prezentă în lucrarea originală numită *CINDEX*.

4.1.2 Regiștrii

Regiștrii mașinii WAM se împart în două categorii:

- de uz general
- specializați

Regiștrii de uz general

Regiștrii de uz general au lățimea cât a unui cuvânt a memoriei și sunt împărțiți logic în următoarele categorii:

- regiștrii de variabilă - folosiți pentru manipularea termenilor când se construiesc sau se unifică. Ei se notează cu $X(i)$ unde X - indică registru de variabilă și i - indică numărul registrului.
- regiștrii de argument - alocați sistematic pentru pregătirea argumentelor a următorului apel de regulă sau predicat. Ei se notează cu $A(i)$ unde A - indică registru de argument și i - indică numărul registrului.

NOTA BENE - A și X sunt doar niște notații pentru a indica contextul în care un registru este folosit. $A(1) = X(1)$

Regiștrii specializați

WAM folosește o serie de regiștrii specializați care indică starea procesorului la un moment dat de timp. În continuare se vor prezenta numele lor și o descriere sumară a rolului pe care îl îndeplinesc.

- *Registrul P* - folosit să indice adresa instrucțiunii curente.
- *Registrul CP* - folosit să indice adresa instrucțiunii care urmează să fie executată la întoarcerea cu succes din apelul unei proceduri.
- *Registrul S* - folosit ca registru de unificare, reține adresa din zona de memorie HEAP a unui termen.
- *Registrul Mode* - folosit pentru a indica modul de execuție, read sau write.
- *H* - folosit pentru a reține adresa vârfului zonei de memorie HEAP.

- *HB* - folosit pentru a reține adresa vârfului zonei de memorie HEAP la timpul ultimului punct de alegere.
- *B0* - registru de tăietură.
- *B* - folosit pentru a reține adresa începutului punctului de alegere anterior.
- *E* - folosit pentru a reține adresa mediului local.
- *TR* - folosit pentru a indica vârful zonei de memorie TRAIL.

Din cauza că FPWAM suportă doar un subset al limbajului Prolog, registrul B0 care este folosit pentru tăieturi nu este prezent în implementare.

4.1.3 Reprezentarea termenilor

În continuare se va prezenta modalitatea de reprezentare a termenilor Prolog în WAM, respectiv FPWAM. Pentru stocarea lor vom folosi un bloc global din zona de memorie numită HEAP. FPWAM va suporta patru tipuri de termeni, structuri, variabile, constante și liste. Fiecare din acești termeni va avea un marcaj unic pentru a putea fi discriminați pe HEAP.

Reprezentarea structurilor

Structurile sunt niște termeni non-variabili. Pentru a reprezenta o structură de tipul $f(t_1, \dots, t_n)$, vor fi folosite $n + 2$ cuvinte de memorie. Primul va fi un poantor către adresa celui de-al doilea care indică termenul f/n . Evident aceste două cuvinte nu necesită să fie la adrese consecutive. Celelalte n cuvinte vor fi folosite pentru a reține referințe către rădăcinile subtermenilor $t_1 \dots t_n$. Mai specific, primul cuvânt va fi de forma $\langle STR, a \rangle$, unde *STR* reprezintă marcajul și *a* reprezintă adresa la care se află reprezentarea lui f/n . Începând de la adresa *a* se va afla întodeauna un bloc contiguu de $n+1$ cuvinte ce reprezintă numele/aritatea structurii și cele n referințe către subtermenii $t_1 \dots t_n$.

Reprezentarea variabilelor

O variabilă va fi indentificată printr-un poantor și va fi reprezentat folosind doar un cuvânt din memorie, adică o celulă variabilă. O asemenea celulă va fi indentificată prin $\langle REF, a \rangle$ unde *REF* denotă marcajul unei referințe și *a* adresa către locația de memorie spre care se face referire. Prin convenție, o variabilă este nelegată dacă ea pointează către ea însuși.

Reprezentarea constantelor

O constantă va fi reprezentată folosind doar un cuvânt din memorie. O asemenea celulă va fi indentificată prin $\langle CON, c \rangle$, unde *CON* reprezintă marcajul, iar *c* reprezintă valoarea constantei.

0	$h/2$	
1	REF	1
2	REF	2
3	$f/1$	
4	REF	2
5	$p/3$	
6	REF	1
7	STR	0
8	STR	3

Figura 4.1: Reprezentare a termenului $f(Z, h(Z, W), f(W))$

Reprezentarea listelor

Cele mai întâlnite structuri de date în programele Prolog sunt listele, fapt pentru care se vor introduce marcaje speciale pentru acestea. O element dintr-o listă este indentificat prin perechea de celule $\langle LIS, a \rangle$, unde LIS reprezintă marcajul prin care se discriminează lista, iar celula de la adresa a unde se găsește referința către data elementului din listă. Pentru a respecta contiguitatea termenilor, aceste prerechi trebuie să fie la adrese consecutive în memorie până la terminarea listei care este indicată prin constanta $\langle CON, NIL \rangle$.

Exemple de reprezentări ai termenilor

În 4.1 se arată reprezentarea termenului $f(Z, h(Z, W), f(W))$ pe HEAP. În acest exemplu se demonstrează reprezentarea variabilelor nelegate și a structurilor. De observat aici este contiguitatea subtermenilor a structurilor, cât și nerespectarea formatului de $n + 2$ celule. Această mică optimizare se întâmplă doar când faptul urmează să fie apelat dintr-o regulă, celulele de tip $\langle STR, a \rangle$ aflându-se în regiștrii de argument. Această optimizare va fi discutată în capitolele următoare. În toate celelalte cazuri, celulele anterior amintite vor exista pe HEAP.

În 4.2 se demonstrează reprezentarea unei liste $[_, _, f(b), a]$, în care toate primele două elemente sunt niște variabile nelegate iar celelalte un termen structurat și o constantă. De observat în acest exemplu este contiguitatea listei care se termină prin constanta specială $\langle CON, NIL \rangle$ și existența celulei $\langle STR, _ \rangle$.

4.1.4 Stiva-AND și Stiva-OR

Pentru a suporta mediile locale și mecanismul de backtracking, în arhitectura WAM există două tipuri de stive. Chiar dacă ele sunt diferite ca principiu, cadrele lor se află intercalate în aceeași memorie fizică din motive care vor fi discutate ulterior.

0	LIS	1
1	REF	1
2	LIS	3
3	REF	3
4	LIS	5
5	STR	10
6	LIS	7
7	CON	a
8	CON	NIL
9	STR	10
10	$f/1$	
11	CON	b

Figura 4.2: Reprezentarea unei liste $[_, _, f(b), a]$

E	CE (mediul de revenire)
E+1	CP (adresa de revenire)
E+2	n (numărul variabilelor permanente)
E+3	Y_1 (variabilă permanentă)
...	
E+n+2	Y_n (variabilă permanentă)

Figura 4.3: Structura unui cadru din stiva-AND

Stiva-AND

Stiva-AND sau stiva mediilor locale este folosită ca mediu de activare a procedurilor. Așa cum este schițat în 4.3, un cadru în această stivă conține:

- adresa stivei a precedentului mediu local (cel la care va trebui să revenim în caz de success)
- adresa următoarei instrucțiuni pe care trebuie să o executăm în caz de success
- numărul de variabile permanente folosite de către această procedură
- n celule de date pentru variabilele permanente (posibil 0)

Stiva-OR

Pentru a permite operația de backtracking avem nevoie de un spațiu pentru a putea salva starea mașinii la fiecare apel de procedură care oferă alternative. În caz de eșec, la încercarea unei alternative este necesară refacerea stării mașinii la acel moment de timp. Așa cum este schițat în -ref, un cadru al unui punct de alegere conține:

B	n (aritatea)
B+1	A_1 (registrul de argument 1)
...	
B+n	A_n (registrul de argument n)
B+n+1	CE (mediul de revenire)
B+n+2	CP (adresa de revenire)
B+n+3	B (precedentul cadru de alegere)
B+n+4	B (următoarea clauză care urmează să fie executată)
B+n+5	TR (valoarea registrului TR)
B+n+6	H (valoarea registrului H)

Figura 4.4: Structura unui cadru din stiva-OR

- n - aritatea acestei proceduri
- regiștrii de argument $A_1 \dots A_n$ - ei trebuie reținuți deoarece vor fi suprascriși de alte proceduri
- mediul local (valoarea registrului E) - pentru recuperarea mediului local
- adresa următoarei instrucțiuni pe care trebuie s-o executăm la întoarcerea din procedură (valoarea registrului CP)
- ultimul cadru de alegere în cazul în caz de eșec (valoarea registrului B)
- adresa unde se află începutul instrucțiunilor pentru următoarea clauză care să fie executată
- vârful stivei $TRAIL$ (valoarea registrului TR) - pentru a efectua operația auxiliară $unwind_{trail}$ la revenirea din backtracking. Aceasta operație va dezlega variabilele care au fost legate după crearea acestui punct de alegere.
- vârful stivei $HEAP$ (valoarea registrului H) - pentru a recupera spațiul ocupat de termenii creați în timpul încercării eșuate care va rezulta prin întoarcerea la acest punct de alegere.

Protejarea mediilor locale

Motivul pentru care se folosește aceeași memorie fizică pentru ambele stive este că logica pentru protejarea mediilor locale este intuitivă și simplă. Dacă se alocă spațiu pe stivă pentru un mediu local și se întâlnește ulterior un punct de alegere, mediul local este protejat de suprascriere de către cadrul punctului de alegere chiar dacă este dealocat. Astfel, în caz de eșec, la revenirea la acel punct de alegere, mediul se recuperează fără ca datele să fie alterate. Mai specific, orice cadru alocat trebuie să fie amplasat peste ultimul cadru de alegere.

4.1.5 Apelul de proceduri

Spre deosebire de mașina WAM care suportă adăugarea de fapte în momentul rulării, FPWAM nu își propune să trateze acest lucru. Astfel este necesară cunoașterea tuturor faptelor și regulilor la momentul compilării. Instrucțiunile originale de control a mașinii WAM vor fi adaptate relaxării acestei constrângeri, nefiind nevoie să se mai testeze existența predicatului în baza de fapte și reguli Prolog. Ele se vor explica în -ref.

4.1.6 Indexarea codului

În cazul în care există mai multe puncte de alegere, o metodă naivă ar fi de a trece prin toate până ce una dintre ele s-ar finaliza cu succes. Această metodă este costisitoare din punct de vedere a timpului, deoarece impune o căutare liniară prin toate punctele de alegere. Aceasta din urmă este folosită doar când arugmentul predicatului este o variabilă nelegată, fiind necesar să încercăm toate opțiunile. Pentru a rezolva această problemă s-a găsit o soluție parțială care se comportă bine în practică, bazată pe tendința programatorilor să discrimineze predicatele după primul argument. Astfel s-au introdus trei nivele de indexare a codului:

- Prin primul nivel se grupează faptele sau regulile după tipul termenului primului argument. De exemplu, dacă se apelează un predicat având primul argument o listă, se va face salt la grupul în care se află doar predicatele care se unifică cu o listă, testându-le pe rând. După cum am amintit mai sus, în cazul unei variabile se vor parcurge toate posibilitățile. Pentru a face uz de această indexare, se folosește instrucțiunea *switch_on_term* care va fi explicată mai târziu.
- De cele mai multe ori indexarea bazată doar pe tipul termenului primului argument nu este suficientă în practică. Ce se întâmplă dacă în baza de predicate se găsesc $e(1, \dots)$ până la $e(1000000, \dots)$ și se apelează predicatul $e(1000000, \dots)$? Instrucțiunea *switch_on_term* își face datoria și ghidează execuția programului către blocul unde se află predicatele grupate după constate, dar aici WAM trebuie să le testeze pe rând până când una dintre se unifică cu succes. Această căutare liniară ar fi foarte costisitoare și în consecință s-a introdus un al doilea nivel de indexare. În specificația WAM originală se utilizează tabele de dispersie de dimensiune N pentru fiecare predicat având forma $\{c_i : L_c i\}$, în care fiecare c_i este cheie pentru numărul instrucțiunii unde se află codul predicatului cu care s-ar unifica. În cazul în care constanta nu se găsește în tabelul de dispersie, se execută backtracking. Această optimizare reduce timpul căutării la $O(c)$. Deoarece FPWAM nu suportă predicate dinamice, s-a introdus o nouă zonă de memorie *CINDEX* care este populată la fiecare consultare a unei surse Prolog. Structura unei celule din zona de memorie *CINDEX* este formată din concatenarea constantei și adresa instrucțiunii unde se află codul predicatului. Aceste celule sunt sortate crescător după valoarea constantei pentru fiecare predicat. Aplicând algoritmul de căutare binară, în FPWAM această căutare va avea complexitatea $O(\log(n))$. Instrucțiunea modificată care face acest lucru se găsește la ref.

- Ultimul nivel de indexare este folosit să grupeze predicatele care au aceeași valoare a constatelor.

4.1.7 Setul abstract de instrucțiuni

Cea mai bună modalitate de a înțelege un procesor este de a-i cunoaște instrucțiunile și cum ele operează. Din cauza faptului că arhitectura WAM conține un minimum de 30 de instrucțiuni, în acest subcapitol se va încerca atingerea tuturor tipurilor de instrucțiuni cu exemple și explicații relevante din fiecare categorie. Deoarece FPWAM nu surpotă instrucțiuni de tăietură, ele nu vor fi explicate în acest subcapitol.

Instrucțiunile mașinii WAM se încadrează în 7 categorii:

1. instrucțiuni de tipul PUT
2. instrucțiuni de tipul GET
3. instrucțiuni de UNIFICARE
4. instrucțiuni de CONTROL
5. instrucțiuni de ALEGERE
6. instrucțiuni de INDEXARE
7. instrucțiuni de TĂIETURĂ

Pentru a simplifica instrucțiunile se va folosi notația V_n care poate reprezenta ori un registrul cu numărul n , ori variabila de pe stivă cu numărul n .

4.1.8 Instrucțiuni de tipul PUT

Tipul acesta de instrucțiune este responsabil să încarce argumentele capetelor următoarelor reguli sau fapte, în regiștrii de argument A_i .

- *put_variable* X_n, A_i

Împinge o variabilă nelegată pe vârful zonei HEAP și copiaz-o în regiștrii X_n și A_i .

$$\begin{aligned} \text{HEAP}[H] &\leftarrow \langle \text{REF}, H \rangle \\ X_n &\leftarrow \text{HEAP}[H] \\ A_i &\leftarrow \text{HEAP}[H] \\ H &\leftarrow H + 1 \\ P &\leftarrow P + 1 \end{aligned}$$

- *put_variable* Y_n, A_i

Inițializează variabila n de pe stivă din mediul curent la o valoare nelegată și setează registrul A_i să poarte către adresa ei.

$$\begin{aligned} addr &\leftarrow E + n + 2 \\ STACK[addr] &\leftarrow \langle REF, addr \rangle \\ A_i &\leftarrow STACK[addr] \\ P &\leftarrow P + 1 \end{aligned}$$

- *put_unsafe_value* Y_n, A_i

Dacă adresa dereferențiată variabilei Y_n este legată în mediul current, globalizează variabila prin împingerea uneia noi în zona de memorie HEAP și legarea ei cu Y_n . În caz contrar setează A_i la valoarea variabilei Y_n .

$$\begin{aligned} addr &\leftarrow deref(E + n + 2) \\ \textbf{if } addr < E \textbf{ then} \\ &A_i \leftarrow STACK[addr] \\ \textbf{else} \\ &HEAP[H] \leftarrow \langle REF, H \rangle \\ &bind(addr, H) \\ &A_i \leftarrow HEAP[H] \\ &H \leftarrow H + 1 \\ \textbf{end if} \\ P &\leftarrow P + 1 \end{aligned}$$

- *put_structure* $f/n, A_i$

Împinge o celulă în zona de memorie HEAP care conține functorul f/n și setează registrul A_i la o celulă STR care poartă către zona de memorie a functorului.

$$\begin{aligned} HEAP[H] &\leftarrow f/n \\ A_i &\leftarrow \langle STR, H \rangle \\ H &\leftarrow H + 1 \\ P &\leftarrow P + 1 \end{aligned}$$

Instrucțiuni de tipul GET

Instrucțiunile de tipul GET corespund argumentelor capetelor clauzelor. Ele sunt responsabile să potrivească argumentele clauzei cu cele date în regiștrii de argument.

- *get_variable* V_n, A_i

Setează valoarea variabilei V_n cu cea din registrul A_i

$$\begin{aligned} V_n &\leftarrow A_i \\ P &\leftarrow P + 1 \end{aligned}$$

- *get_value* V_n, A_i

Unifică variabila V_n cu registrul A_i . În caz de eșuare execută operația de backtracking.

```

unify( $V_n, A_i$ )
if fail then
    backtrack
else
     $P \leftarrow P + 1$ 
end if

```

- *get_structure* f_n, A_i

Dacă valoarea dereferențiată a registrului A_i este o variabilă nelegată, leagă acea variabilă de o nouă structură f/n alocată pe HEAP și setează registrul *mode* la valoarea *write*. În caz contrar dacă în memorie se găsește aceeași structură, setează registrul *S* la locația functorului f/n și setează registrul *mode* la valoarea *read*. În alte cazuri, execută *backtrack*.

```

 $addr \leftarrow deref(A_i)$ 
switch STORE[ $addr$ ] do
    case  $\langle REF, \_ \rangle$ 
         $HEAP[H] \leftarrow \langle LIS, H + 1 \rangle$ 
        bind( $addr, H$ )
         $H \leftarrow H + 1$ 
        mode  $\leftarrow write$ 
    case  $\langle LIS, a \rangle$ 
         $S \leftarrow a$ 
        mode  $\leftarrow read$ 
    case other
        fail  $\leftarrow true$ 
if fail then
    backtrack
else
     $P \leftarrow P + 1$ 
end if

```

Instrucțiuni de unificare

Aceste instrucțiuni corespund argumentelor unei structuri sau a unei liste fiind responsabile cu unificarea a termenilor existenți în modul *read* sau crearea de noi termeni și structuri în modul *write*.

- *unify_variable* V_n

În modul *read*, plasează conținutul memoriei de la adresa S în variabila V_n . În modul *write*, împinge o variabilă nelegată pe vârful memoriei HEAP și copiează-o în V_n . În ambele moduri, incrementează registrul S cu unu.

```

switch mode do
  case read
     $V_n \leftarrow \text{HEAP}[S]$ 
  case write
     $\text{HEAP}[H] \leftarrow \langle \text{REF}, H \rangle$ 
     $V_n \leftarrow \text{HEAP}[H]$ 
     $H \leftarrow H + 1$ 
 $S \leftarrow S + 1$ 
 $P \leftarrow P + 1$ 

```

- *unify_value* V_n

În modul *read*, unifică variabila V_n cu conținutul memoriei HEAP de la adresa S . În modul *write*, împinge o variabilă aflată în V_n pe vârful memoriei HEAP. În ambele moduri, incrementează registrul S cu unu.

```

switch mode do
  case read
    unify( $V_n, S$ )
  case write
     $\text{HEAP}[H] \leftarrow V_n$ 
     $H \leftarrow H + 1$ 
 $S \leftarrow S + 1$ 
if fail then
  backtrack
else
   $P \leftarrow P + 1$ 
end if

```

Instrucțiuni de control

Aceste instrucțiuni sunt folosite pentru a dicta mersul programului în execuție cât și alocării și dealocării cadrelor locale unei proceduri.

- *allocate* N

Alocă un nou mediu local pe stivă, setându-i ultimul adresa ultilui cadru de mediu, adresa instrucțiunii de revenire din apelul curent și numărul variabilelor permanente.

if $E > B$ **then**

$nouE \leftarrow E + STACK[E + 2] + 3$

else

$nouE \leftarrow B + STACK[B] + 7$

end if

$STACK[nouE] \leftarrow E$

$STACK[nouE + 1] \leftarrow CP$

$STACK[nouE + 2] \leftarrow N$

$E \leftarrow nouE$

$P \leftarrow P + 1$

- *deallocate*

Șterge mediul current prin resetarea registrului E și registrului CP .

$CP \leftarrow STACK[E + 1]$

$E \leftarrow STACK[E]$

$P \leftarrow P + 1$

- *call* P, N

Șterge mediul current prin resetarea registrului E și registrului CP .

$CP \leftarrow STACK[E + 1]$

$E \leftarrow STACK[E]$

$P \leftarrow P + 1$

- *proceed*

Continuă execuția cu instrucțiunea de la adresa indicată de către registrul CP .

$P \leftarrow CP$

Instrucțiuni de alegere

Aceste instrucțiuni sunt destinate exclusiv mecanismului de backtracking. Ele salvează și refac starea procesorului prin alocarea cadrelor pe stiva-OR și controlează execuția programului în cazul în care există mai multe puncte de alegere.

- *try_me_else* L

Alocă un nou cadru de alegere pe stivă, setându-i adresa următoarei clauze la L , iar celelate câmpuri în concordanță cu contextul curent, și setează registrul B să poarte către noul cadru.

```

if  $E > B$  then
     $nouB \leftarrow E + STACK[E + 2] + 3$ 
else
     $nouB \leftarrow B + STACK[B] + 7$ 
end if
 $STACK[nouB] \leftarrow aritatea$ 
 $n \leftarrow aritatea$ 
for  $i < n$  do
     $STACK[nouB + i] \leftarrow A_i$ 
end for
 $STACK[nouB + n + 1] \leftarrow E$ 
 $STACK[nouB + n + 2] \leftarrow CP$ 
 $STACK[nouB + n + 3] \leftarrow B$ 
 $STACK[nouB + n + 4] \leftarrow L$ 
 $STACK[nouB + n + 5] \leftarrow TR$ 
 $STACK[nouB + n + 6] \leftarrow H$ 
 $B \leftarrow nouB$ 
 $HB \leftarrow H$ 
 $P \leftarrow P + 1$ 

```

• *retry_me_else L*

La revenirea din backtracking la această instrucțiune, resetează starea procesorului cu cea salvată în cadru.

```

 $n \leftarrow STACK[B]$ 
for  $i < n$  do
     $A_i \leftarrow STACK[B + i]$ 
end for
 $E \leftarrow STACK[B + n + 1]$ 
 $CP \leftarrow STACK[B + n + 2]$ 
 $STACK[B + n + 4] \leftarrow L$ 
 $unwind\_stack(STACK[B + n + 5], TR)$ 
 $TR \leftarrow STACK[B + n + 5]$ 
 $H \leftarrow STACK[B + n + 6]$ 
 $HB \leftarrow H$ 
 $P \leftarrow P + 1$ 

```

• *trust_me*

La revenirea din backtracking la această instrucțiune, resetează starea procesorului cu cea salvată în cadru, apoi șterge-l resetând registrul B cu predecesorul lui.

```

 $n \leftarrow STACK[B]$ 
for  $i < n$  do
     $A_i \leftarrow STACK[B + i]$ 
end for
 $E \leftarrow STACK[B + n + 1]$ 
 $CP \leftarrow STACK[B + n + 2]$ 
 $unwind\_stack(STACK[B + n + 5], TR)$ 
 $TR \leftarrow STACK[B + n + 5]$ 
 $H \leftarrow STACK[B + n + 6]$ 
 $HB \leftarrow STACK[B + n + 6]$ 
 $B \leftarrow STACK[B + n + 3]$ 
 $P \leftarrow P + 1$ 

```

Instrucțiuni de indexare

După cum s-a discutat în capitolele anterioare, aceste instrucțiuni sunt folosite pentru a optimiza timpul de căutare a predicatelor.

- *switch_on_term*

Setează registrul P în funcție de valoarea dereferențiată a registrului A_1 . În celulele $P + 1..4$ se găsesc instrucțiuni de tip control care ghidează programul către blocul de execuție corespunzător sau instrucțiune *fail* în cazul în care nu există predicat care are ca prim argument tipul indicat de valoarea din registrul A_1 .

```

switch  $STORE[deref(A_1)]$  do
    case  $< REF, >$ 
         $P \leftarrow P + 1$ 
    case  $< CON, >$ 
         $P \leftarrow P + 2$ 
    case  $< LIS, >$ 
         $P \leftarrow P + 3$ 
    case  $< STR, >$ 
         $P \leftarrow P + 4$ 

```

- *switch_on_con_str low, high*

Execută căutarea binară a valorii dereferențiate din registrul A_i în zona de memorie $CINDEX$ începând de la adresa low până la adresa $high$. În caz că nu s-a găsit valoarea, backtrack.

```

< tag, val > ← STORE[deref( $A_1$ )]
< found, inst > ←
  binary_search(val, low, high, CINDEX)
if found then
   $P \leftarrow inst$ 
else
  backtrack
end if

```

4.1.9 Operațiile auxiliare

Pe lângă setul de instrucțiuni, arhitectura WAM dispune de niște operații auxiliare încorporate implementate hardware:

- operația $deref(a1)$ - după cum îi sugerează numele, primește o referință și returnează cuvântul din memorie la care se face referire.
- operația $bind(a1, a2)$ - aceasta primește două adrese pe care să le lege împreună. Operația de legare se execută în funcție de cuvintele aflate la adresele date, respectând principiile WAM-ref.
- operația $trail(a1)$ - stabilește dacă adresa primită ca parametru să fie împinsă în zona de memorie $TRAIL$
- operația $unwind_trail(a1, a2)$ - dezleagă toate adresele aflate în zona de memorie $TRAIL$ începând cu adresa $a1$ până la $a2$
- operația $unify(a1, a2)$ - încearcă să unifice cei doi termeni aflați la adresele $a1$ și $a2$. Algoritmul este bazat pe metoda UNION/FIND ref -[AHU74]

4.2 Soluția programelor software

4.2.1 Compilarea PROLOG-WAM

Un principal obiectiv a acestui proiect a fost construirea procesorului astfel încât să fie compatibil cu cel puțin unul dintre compilatoarele Prolog existente care sunt bazate pe WAM. Datorită faptului că GNU PROLOG este o distribuție cu sursă deschisă și are suport încorporat de compilare PROLOG → WAM, alegerea a fost evidentă.

4.2.2 Translatarea din WAM în cod mașină FPWAM

Pentru a face legătura între compilatorul GNU și procesorul FPWAM este nevoie de un program translator. Rolul acestui program este de a parsa și de a traduce instrucțiunile WAM în cod mașină, cu scopul de a fi înțelese de către procesor. Acest procesul nu este unul simplu, pe parcursul executării sale fiind întâmpinate numeroase probleme. În continuare, ele vor fi prezentate împreună cu modul de soluționare abordat.

- Spre deosebire de Prolog, mașina FPWAM folosește cuvinte de o lățime fixă pentru a reprezenta constantele, numele structurilor, faptelor și a predicatelor. În urma parsării instrucțiunilor, toți acești termeni vor fi reprezentați sub formă textuală, formatul fiind nepotrivit mașinii FPWAM. Pentru a soluționa această problemă se vor folosi două dicționare, unul pentru constante, iar celălalt pentru structuri și predicate. În aceste dicționare se vor reține codificarea termenilor într-o reprezentare unică care poate fi manipulată de către FPWAM. Intrarea unui asemenea dicționar este de forma $str_i : c_i$, unde cheia este str_i reprezentată de un șir de caractere și c_i codificarea asociată ei. Pentru a menține unicitatea valorilor c_i , există un contor pentru fiecare dintre cele două dicționare care se incrementează la adăugarea unei noi intrări.
- Din cauza discrepanței dintre Prolog și limitarea impusă de către FPWAM ca orice predicat să fie definit la momentul compilării, pot să se genereze instrucțiuni *nerezolvate*. Aceste instrucțiuni apar ori în situațiile în care ele apelează un predicat al cărui definiție nu a fost întâlnită până la momentul parsării, ori în cazul instrucțiunilor care fac salt la o etichetă neprocesată. Neștiind adresa instrucțiunii de apel sau salt, aceste instrucțiuni rămân *nerezolvate*. Problema își găsește soluția prin:
 - folosirea a două dicționare pentru reținerea relațiilor $\{predicat : adresă\}$ și a relațiilor $\{etichetă : adresă\}$
 - reținerea tuturor instrucțiunilor *nerezolvate*

După terminarea parsării, se reiau instrucțiunile *nerezolvate* și se rezolvă prin căutarea în dicționarele amintite, adresele de apel sau salt.

4.2.3 Mediul de executare

Pentru a putea compila întrebări Prolog și a vizualiza rezultate, mediul de executare folosește rezultatele intermediare ale translatorului și compilatorului. Aceste rezultate intermediare sunt:

- Dicționarele de constante și structuri/predicate ale translatorului. Ele sunt necesare pentru a face decodificarea cuvintelor WAM în forma lor textuală și pentru codificarea noilor întrebări Prolog.
- Dicționarul care reține relația $\{predicat : adresă\}$ pentru compilarea întrebărilor Prolog.

Capitolul 5

Proiectare de Detaliu și Implementare

5.1 Arhitectura generală a sistemului

Sistemul construit este format din două module principale:

- modulul software, care are datoria să genereze cod WAM prin folosirea compilatorului GNU PROLOG, apoi să-l translateze în cod mașină și să-l trimită către procesorul FPWAM. Acest modul va rula pe calculatorul gazdă.
- modulul hardware, care are datoria să execute programele Prolog compilate de către modulul software.

După cum se poate observa și în 5.1, aceste două module comunică prin protocolul UART.

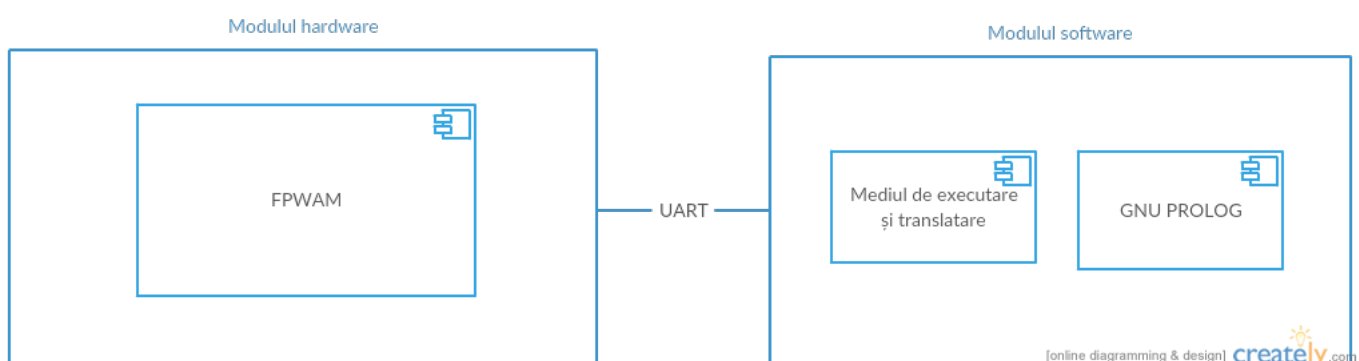


Figura 5.1: Arhitectura generală a sistemului

5.2 Modulul hardware

5.2.1 Pachetul FpwamPkg.vhd

Din cauza faptului că implementarea FPWAM este destul de stufoasă, ar fi fost destul de greu de urmărit semnale care sunt formate doar din 1 și 0. S-a luat decizia de a crea un pachet special care să conțină definiția tuturor constantelor, tipurilor personalizate de semnal, funcțiilor și instrucțiunilor implementate care au legătură cu FPWAM. Totodată prin utilizarea acestui pachet se permite schimbarea configurației procesorului doar prin modificarea valorilor unor constante și permite extinderea funcționalității procesorului prin simpla adăugare a unor noi instrucțiuni. Se recomandă utilizarea funcțiilor definite în acest pachet, deoarece ele iau în considerare modificarea constantelor. În continuare se vor prezenta cele mai importante elemente din acest pachet.

Constante

- *kWamAddressWidth* - această constantă determină adâncimea memoriei BlockRAM din zona HEAPSTACK
- *kWamWordWidth* - această constantă determină lățimea cuvintelor cu care procesorul FPWAM lucrează. Cu cât este mai mare această constantă, cu atât crește numărul de termeni pe care FPWAM poate să-i manipuleze. Atenție! Prin modificarea acestei constante, crește și lățimea memoriilor care rețin termeni WAM.
- *kWamPdlAddressWidth* - această constantă determină adâncimea memoriei PDL din unitatea de unificare.
- *kWamTrailAddressWidth* - această constantă determină adâncimea memoriei TRAIL.
- *kGPRAddressWidth* - această constantă determină adâncimea setului de regiștrii generali.
- *kFunctorWidth* - această constantă determină numărul de biți de reprezentarea a functorului dintr-un cuvânt FPWAM.
- *kArietyWidth* - această constantă determină aritatea maximă a unui fapt, structuri sau reguli. De obicei ea este egală cu constanta care determină numărul regiștrilor.
- *kWamInstructionWidth* - această constantă determină lățimea unei instrucțiuni FPWAM.
- *kWamInstrMemWidth* - această constantă determină adâncimea maximă a memoriei de instrucțiuni.
- *kWamHeapStart* - această constantă determină adresa de start în memoria HEAPSTACK a secțiunii HEAP.

- *kWamStackStart* - această constantă determină adresa de start în memoria HEAPSTACK a secțiunii STACK.
- *kInstrDecodeWidth* - această constantă determină lățimea cuvântului prin care se reprezintă codul unei instrucțiuni.

Tipuri de semnale

Multe tipuri a semnalelor din acest pachet nu au o importanță semnificativă, multe fiind utilizate ca intrări de control ale multiplexoarelor din entitatea TopLevel și nu vor fi prezentate în această lucrare. Cele mai importante din punct de vedere a dezvoltării ulterioare sunt:

- tipul enumerat *tag_t* - în acest tip se definesc etichetele obiectelor suportate de către mașina FPWAM. În acest moment FPWAM suportă următoarele obiecte:
 - *tag_str_t* - etichetă pentru termeni structurați
 - *tag_ref_t* - etichetă pentru referințe
 - *tag_lis_t* - etichetă pentru listă
 - *tag_int_t* - etichetă pentru constante

Un cuvânt etichetat în FPWAM este format din etichetă|valoarea cuvântului.

- tipul enumerat *instruction_t* - în acest tip se definesc instrucțiunile implementate de către FPWAM. Setul de instrucțiuni curent suportat este următorul:
 1. *i_nop*
 2. *i_put_structure_t* -- 000001
 3. *i_put_variable_X_t* -- 000010
 4. *i_put_variable_Y_t* -- 000011
 5. *i_put_value_t* -- 000100
 6. *i_get_structure_t* -- 000101
 7. *i_get_variable_t* -- 000110
 8. *i_get_value_t* -- 000111
 9. *i_unify_variable_t* -- 001000
 10. *i_unify_value_t* -- 001001
 11. *i_call_t* -- 001010
 12. *i_proceed_t* -- 001011
 13. *i_allocate_t* -- 001100
 14. *i_deallocate_t* -- 001101

- 15. i_try_me_else_t -- 001110
- 16. i_retry_me_else_t -- 001111
- 17. i_trust_me_t -- 010000
- 18. i_put_unsafe_value_t -- 010001
- 19. i_put_list_t -- 010010
- 20. i_put_constant_t -- 010011
- 21. i_get_list_t -- 010100
- 22. i_get_constant_t -- 010101
- 23. i_unify_local_value_t -- 010110
- 24. i_unify_constant_t -- 010111
- 25. i_unify_void -- 011000
- 26. i_try_t -- 011001
- 27. i_retry_t -- 011010
- 28. i_trust_t -- 011011
- 29. i_execute_t -- 011100
- 30. i_unify_structure_t -- 011101
- 31. i_switch_on_term_t -- 011110
- 32. i_fail_t -- 011111
- 33. i_switch_on_int_str_t -- 100000
- 34. i_unify_list_t -- 100001

Formatul unei instrucțiuni este de trei tipuri:

1. |COD_INSTR|REG1|CUVÂNT_WAM| - pentru instrucțiuni registru - constantă
2. |COD_INSTR|REG1|REG2| - pentru instrucțiuni registru-registru
3. |COD_INSTR|NULL|CUVÂNT_WAM| - pentru instrucțiuni de unificare, apel sau constantă

Funcții

- function fpwam_tag(word : std_logic_vector) return tag_t; -returnează eticheta cuvântului FPWAM dat ca intrare.
- function fpwam_value(word : std_logic_vector) return std_logic_vector -returnează valoarea cuvântului FPWAM dat ca intrare

- `function fpwam_word(word : std_logic_vector; tag : tag_t) return std_logic_vector`
- creează un cuvânt FPWAM cu valoarea și eticheta dată la intrare
- `function fpwam_functor(word : std_logic_vector(kWamWordWidth -1 downto 0)) return std_logic_vector` - returnează functorul cuvântului dat ca intrare
- `function fpwam_arity(word : std_logic_vector(kWamWordWidth -1 downto 0)) return std_logic_vector` - returnează aritatea unei structuri/fapt date ca intrare
- `function fpwam_instr(word : std_logic_vector(31 downto 0)) return instruction_t`
- returnează tipul de instrucțiune decodificat din vectorul de biți dat ca parametru
- `function fpwam_instr_addr(word : std_logic_vector) return std_logic_vector`
- returnează adresa de apel din cuvântul de la intrare
- `function fpwam_instr_arity(word : std_logic_vector) return std_logic_vector`
- returnează aritatea cuvântului din instrucțiunea dată ca intrare
- `function fpwam_var_on_stack(word : std_logic_vector) return boolean` - returnează o valoare booleană care indică dacă instrucțiunea se referă la o variabilă locală sau la un registru
- `function fpwam_var_stack_addr(instr : std_logic_vector; E : std_logic_vector) return std_logic_vector` - returnează adresa pe stivă a variabilei din instrucțiunea dată ca intrare.

5.2.2 Entitatea TopLevel.vhd și arhitectura generală

După cum se poate observa în 5.2, în această componentă se descrie într-un mod structural arhitectura procesorului implementat. Ea are rolul de a îmbina și a defini căile de date între componente. Din cauza complexității mărite a acestei componente, schema 5.2 este una sumară care nu cuprinde toate legăturile între componente. Rolul ei este de a evidenția dependența între entități și de a oferi unui posibil dezvoltator o imagine de ansamblu a componentelor prezente în procesor.

5.2.3 Entitatea DataFlowControl.vhd sau DFC

Această componentă este creierul procesorului. Ea pune semnalele de control pe căile de date în funcție de instrucțiunea curentă și starea actuală a procesorului. Datorită faptului că procesorul FPWAM are instrucțiuni complexe și multe dintre ele nu se pot executa într-un singur tact, s-au luat în calcul următoarele tehnici de implementare:

- implementarea unității DFC printr-o unitate microprogramată, asemănător procesoarelor CISC

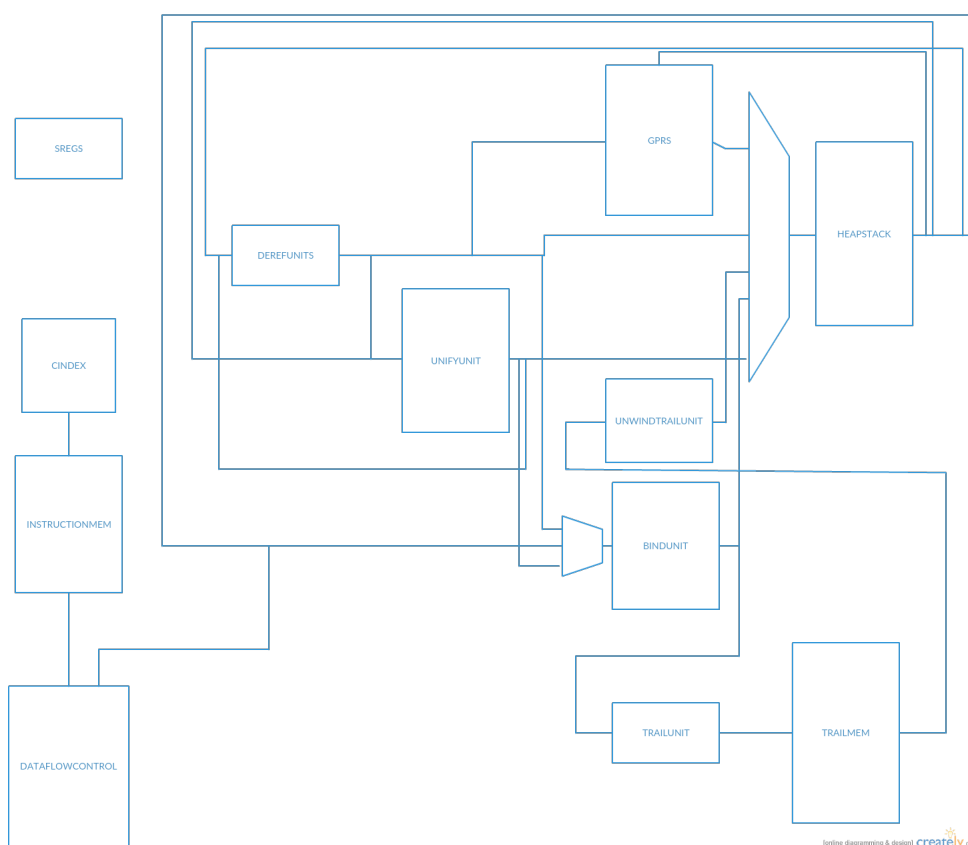


Figura 5.2: Arhitectura generală a procesorului

- implementarea unității DFC printr-un automat de stare de tip *Mealy*.

Chiar dacă avantajele din punct de vedere a performanței sunt mai mari la o unitate microprogramată, s-a ales abordarea automatului de stare deoarece crește lizibilitatea și înțelegerea codului, facilitează depanarea și procesul de adăugare sau modificare a funcționalităților. În momentul de față FPWAM suportă 33 de instrucțiuni, iar automatul din DFC a ajuns la aproximativ 80 de stări. Pentru a minimiza cât de mult penalizările de performanță din punct de vedere a timpului, se va face un compromis și se va folosi codificarea *un bistabil pe stare* (*one – hot encoding*). Chiar dacă 80 de bistabile par mult pentru codificarea stărilor unui automat, resursele din cipul FPGA sunt destul de numeroase.

Din cauza faptului că semnalele de intrare și ieșire a acestei componente sunt de ordinul zecilor, ele nu vor fi prezentate în acest subcapitol. Acest modul este împărțit logic în trei procese principale:

- FSM: `process(clk)` - acest proces se setează pe frontul crescător al semnalului de ceas următoarea stare decodificată
- NEXT_STATE_DECODE: `process(decoded_state, cr_state, deref_done,...)` - acest

proces decodifică următoarea stare a automatului în funcție de starea curentă a automatului și intrările componente.

- `OUTPUT_DECODE: process(cr_state, deref_done, mem_obj, ...)` - setează semnalele de control pe căile de date în funcție de starea curentă a automatului și intrările componente.

Pentru a adăuga stări automatului, trebuie să se adauge noi valori în tipul enumerat `state_t` și să se adauge logica corespunzătoare în procesele `OUTPUT_DECODE` și `NEXT_STATE_DECODE`.

5.2.4 Entitatea GPR.vhd sau General Purpose Registers

Această entitate reprezintă regiștrii de argument și de variabilă prezentați în capitolul Fundamentare Teoretică. Din setul de instrucțiuni abstracte reșee că ei sunt printre cele mai folosite componente a acestui procesor. Datorită numărului mic, ei se vor implementa ca niște memorii RAM distribuite cu două porturi și citire asincronă. O ilustrare grafică a componente se găsește la 5.3.

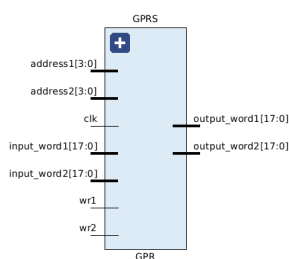


Figura 5.3: Componenta GPR

5.2.5 Entitatea Memory.vhd

Toate zonele de memorie din modelul abstract sunt implementate ca entități separate de BlockRAM, în afară de zona de memorie HEAP și STACK care împărtășesc aceeași instanță fizică. Aceasta a oferit algoritmului de plasare libertatea de a pune componentele care folosesc unele zone de memorie în apropierea lor. Totodată acest artificiu a crescut gradul de paralelizare a instrucțiunilor cât și ușurința personalizării și testării componentelor. Componentele fizice care se bazează pe *Memory.vhd* sunt:

- *TopLevel/INSTRMEM*
- *TopLevel/HEAPSTACK*
- *TopLevel/TRAIL*
- *TopLevel/BMEM* - sau CINDEX

- *TopLevel/UNIFYU/PDLIST*

La implementarea *Memory.vhd* s-au luat în calcul cele două moduri de funcționare a BlockRAM:

- latența citirii să fie de 1 tact
- latența citirii să fie de 2 tacti.

În afară de latența introdusă, cele două abordări au niște efecte subtile care pot scăpa ochiului neexperimentat. Alegerea cea mai convenabilă pare să fie prima opțiune deoarece latența este minimă. Dezavantajul acestei abordări este întârzierea datelor la portul de ieșire, apariția lor fiind de obicei la sfârșitul perioadei de ceas. În cazul în care există cale de la portul de ieșire al memoriei către alte circuite combinaționale care au multe nivele de logică, semnalul nu are timp să se propage înaintea începerii unui nou ciclu de ceas, rezultând în neatingerea constrângerilor de timp și funcționare deficitară. Cealaltă abordare rezolvă această problemă prin includerea automată la ieșirea memoriei niște regiștrii de ieșire. În cazul nostru această soluție nu este convenabilă deoarece memoria este cea mai folosită componentă a procesorului, iar latența de 2 tacti la fiecare citire ar afecta considerabil performanța. Pentru a soluționa parțial ambele probleme s-a făcut un compromis arhitectural. Memoria a fost descrisă ca un BlockRAM cu latență 1, iar în cazurile în care calea combinațională este prea lungă, se pun regiștrii intermediari pentru a salva ieșirile memoriei, rezultatul fiind ca a unei memorii cu latență 2. Totodată pentru creșterea performanței s-a optat ca memoria să aibă două porturi de citire și scriere. În final modul de funcționare a memoriei este descris în 5.4. Adâncimile și lățimile memoriei sunt configurabile prin porturile generice ale entității. O ilustrare grafică a componentei se găsește la 5.5.

Module Name	RTL Object	PORT A (Depth x Width)	W	R	PORT B (Depth x Width)	W	R	Ports driving FF	RAMB18	RAMB36
Memory_behavioral	BRAM_reg	64 K x 18(WRITE_FIRST)	W	R	64 K x 18(WRITE_FIRST)	W	R	Port A and B	0	36

Figura 5.4: Modul de funcționare

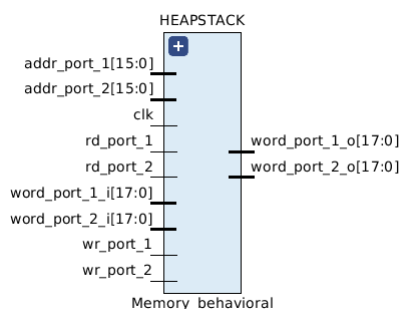


Figura 5.5: Componenta instanțiată a entității *Memory.vhd*

5.2.6 Operațiile auxiliare

În continuare vom trece în revistă operațiile auxiliare WAM. Pentru implementarea lor s-au luat în considerare două abordări:

- implementarea lor în componenta DFC
- implementarea lor ca entități separate în procesorul FPWAM

Prima abordare pare cea mai naturală datorită faptului că în componenta DFC se află logica semnalelor de control care vor fi puse pe căile de date, operațiile auxiliare fiind doar niște rutine comune care se execută pentru mai multe instrucțiuni. După cum s-a prezentat în 5.2.3, componenta DFC este una foarte complexă, având deja peste 80 de stări. Prin adăugarea rutinelor operațiilor auxiliare complexitatea ei ar crește cu aproximativ 25%, acest lucru fiind de nedorit.

A doua abordare chiar dacă pare nenaturală din punct de vedere ingineresc, prezintă mai multe avantaje decât cea anterioară. Implementând fiecare componentă ca un întreg crește modularitatea procesorului și lizibilitatea codului pentru un preț de maxim 2 tacti pe operație auxiliară și maxim 2 nivele de logică în plus pentru multiplexarea detelor de la memorie.

- `DerefUnit.vhd` - Această entitate are rolul de a executa operația de dereferențiere. Construcția ei este tot bazată pe un automat de tip *Mealy*, descrierea lui fiind cu trei procese. Există două asemenea componente în procesorul FPWAM pentru a accelera algoritmul de unificare.
- `BindUnit.vhd` - Această entitate are rolul de a executa operația de legare a două cuvinte FPWAM. Ea este descrisă tot printr-un automat de stări.
- `TrailUnit.vhd` - Această entitate are rolul de a executa operația TRAIL. Ea este singura dintre operațiile auxiliare care este asincronă.
- `UnifyUnit.vhd` - Cea mai complexă dintre operațiile auxiliare. Ea execută algoritmul de unificare dintre două cuvinte FPWAM. În componența ei există o memorie care are rolul structurii *PDL*.
- `UnwindTrailUnit.vhd` - Legată de memoria *TRAIL*, această entitate are rolul de a dezlega toate variabilele începând cu o adresă *a1* până la adresa *a2*.
- `BinarySearch.vhd` - După cum s-a discutat și în Fundamentare Teoretică, această entitate are rolul de executa noua operație de căutare binară în memoria *CINDEX*.

5.3 Modulul software

5.3.1 Arhitectura conceptuală

În 5.6 este prezentată o diagramă UML cu arhitectura modului software. După cum se poate observa, aplicația este împărțită în două blocuri mari:

- blocul dezvoltat în C++ care conține translatorul.
- blocul GPLC, mai specific parser-ul de fișiere .WAM, dezvoltat în C de către Daniel Diaz, care este adaptat în lucrarea curentă pentru a comunica cu translatorul prin API-ul expus.

API-ul amintit se găsește în *GPLCBridge.h, cpp*.

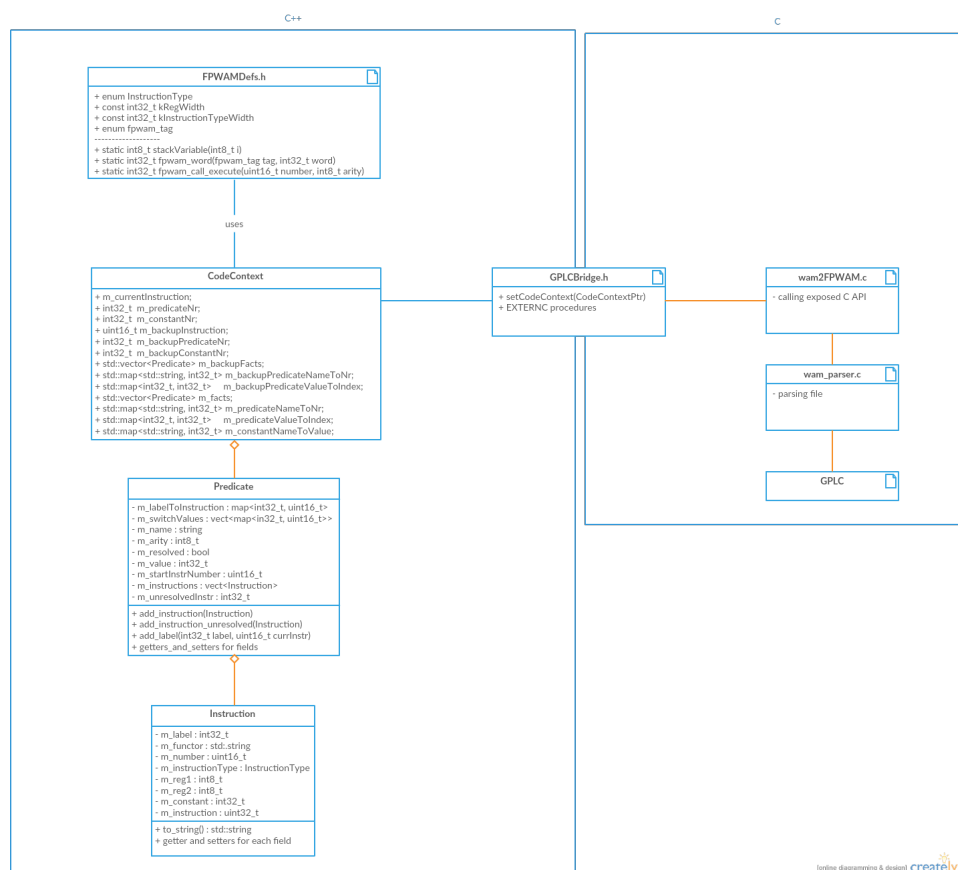


Figura 5.6: Arhitectura modulului software

5.3.2 C API-ul expus al translatorului

Chiar dacă C++ este un superset al limbajului C, un obiect compilat C nu poate fi link-editat cu un obiect compilat C++, chiar dacă el este scris procedural. Această problemă apare din cauza tehnicii de "name mangling" care este diferită la cele două compilatoare. În cazul în care se încearcă link-editarea celor două, cel mai probabil se va arunca o eroare care spune că nu s-au găsit anumite definiții pentru funcția X. Pentru a rezolva această problemă în limbajul C++ există construcția *extern "C"*. Prin plasarea ei în fața funcțiilor și procedurilor îi indicăm compilatorului de C++ să folosească tehnica C de "name mangling". Astfel, link-editarea se va face cu succes.

În cazul de față avem nevoie ca parser-ul WAM să apeleze metodele din clasa *CodeContext* care sunt dedicate fiecărei instrucțiuni. Acest lucru îl vom rezolva prin expunerea acestui obiect prin API-ul C. Pentru a face acest lucru s-au definit două fișiere *GPLCBridge.h*, *cpp*, unde s-au expus și implementat câte o procedură externă C pentru fiecare metodă din clasa *CodeContext* (Vezi 5.7 și 5.8). După cum se poate observa în implementarea acestor proceduri, se ia obiectul din contextul curent și doar se apelează metodele care poartă același nume. Acum nu mai rămâne decât adăugarea funcționalității în blocul C de a apela procedurile expuse în funcție de instrucțiunea parsată.

```
// Created by Bogdan Ardelean on 7/3/17.
//
#ifdef SOFTWARE_GPLCBRIDGE_H
#define SOFTWARE_GPLCBRIDGE_H

#ifdef __cplusplus
#define EXTERNC extern "C"
#else
#define EXTERNC
#endif

typedef void *CodeContextCPtr;
void setCodeContext(CodeContextCPtr codeCtx);

EXTERNC void predicate(const char* name, const int8_t arity);
EXTERNC void get_value(const int8_t XYn, const char c, const int8_t Ai);
EXTERNC void get_variable(const int8_t XYn, const char c, const int8_t Ai);
EXTERNC void get_constant(const char* atom, const int8_t Ai);
EXTERNC void get_list(const int8_t Ai);
EXTERNC void get_structure(const char* name, const int8_t arity, const int8_t Ai);
EXTERNC void put_variableX(const int8_t Xn, const int8_t Ai);
EXTERNC void put_variableY(const int8_t Yn, const int8_t Ai);
```

Figura 5.7: Fișierul GPLCBridge.h

5.3.3 Blocul C și parser-ul de fișiere WAM

wam_parser.h,c

Dezvoltate în totalitate de către Daniel Diaz, aceste două fișiere alcătuiesc parser-ul de fișiere în care se află cod WAM.

5.3.4 wam2FPWAM.h,c

Dezvoltate în mare parte de către Daniel Diaz, aceste două fișiere au fost adaptate să interfațeze cu API-ul expus de către translator. După cum se poate observa în 5.9, apelul *F_put_variable* este redirectionat către funcțiile expuse în API-ul (*put_variableX* sau *put_variableY*) translatorului.

```

//
// Created by Bogdan Ardelean on 7/3/17.
//

#include <iostream>
#include "GPLCBridge.h"
#include "CodeContext.h"

using namespace FPWAM;

static CodeContext* contextObj;

void setCodeContext(CodeContextCPtr codeCtx)
{
    contextObj = static_cast<CodeContext*>(codeCtx);
}

void predicate(const char *name, const int8_t arity)
{
    contextObj->predicate(name, arity);
}

void get_value(const int8_t XYn, const char c, const int8_t Ai)
{
    contextObj->get_value(XYn+1, c, Ai+1);
}

void get_variable(const int8_t XYn, const char c, const int8_t Ai)
{
    contextObj->get_variable(XYn+1, c, Ai+1);
}

```

Figura 5.8: Fișierul GPLCBridge.cpp

```

/*
 * F_PUT_VARIABLE
 *
 */
void
F_put_variable(ArgVal arg[])
{
    Args2(X_Y(xy), INTEGER(a));
    if (c == 'X')
    {
        put_variableX(xy, a);
    }
    else
    {
        put_variableY(xy, a);
    }
}

```

Figura 5.9: Redirecționare apel către API

Capitolul 6

Testare și Validare

Aproximativ 5% din total

6.1 Titlu

6.2 Alt titlu

Capitolul 7

Manual de Instalare și Utilizare

În secțiunea de Instalare trebuie să detaliați resursele software și hardware necesare pentru instalarea și rularea aplicației, precum și o descriere pas cu pas a procesului de instalare. Instalarea aplicației trebuie să fie posibilă pe baza a ceea ce se scrie aici.

În acest capitol trebuie să descrieți cum se utilizează aplicația din punct de vedere al utilizatorului, fără a menționa aspecte tehnice interne. Folosiți capturi ale ecranului și explicații pas cu pas ale interacțiunii. Folosind acest manual, o persoană ar trebui să poată utiliza produsul vostru.

7.1 Titlu

7.2 Alt titlu

Capitolul 8

Concluzii

Cca. 5% din total. Capitolul ar trebui sa conțină (nu se rezumă neapărat la):

- un rezumat al contribuțiilor voastre
- analiză critică a rezultatelor obținute
- descriere a posibilelor dezvoltări și îmbunătățiri ulterioare

8.1 Titlu

8.2 Alt titlu

Bibliografie

- [1] W. Strunk, Jr. and E. B. White, *The Elements of Style*, 3rd ed. Macmillan, 1979.
- [2] E. Bellucci, A. Lodder, and J. Zeleznikow, “Integrating artificial intelligence, argumentation and game theory to develop an online dispute resolution environment.’ in *16th International Conference on Tools with Artificial Intelligence*, 2004, pp. 749–754.
- [3] G. Antoniou, T. Skylogiannis, A. Bikakis, M. Doerr, and N. Bassiliades, “Dr-brokering: A semantic brokering system.’ *Knowledge-Based Systems*, vol. 20, no. 1, pp. 61–72, 2007.
- [4] S. J. Russell, P. Norvig, J. F. Canny, J. M. Malik, and D. D. Edwards, *Artificial intelligence: a modern approach*. Prentice hall Englewood Cliffs, 1995, vol. 2.
- [5] “Ajax tutorial.’ [Online]. Available: <http://www.tutorialspoint.com/ajax/>.

Anexa A

Secțiuni relevante din cod

```
/** Maps are easy to use in Scala. */
object Maps {
  val colors = Map("red" -> 0xFF0000,
                   "turquoise" -> 0x00FFFF,
                   "black" -> 0x000000,
                   "orange" -> 0xFF8040,
                   "brown" -> 0x804000)

  def main(args: Array[String]) {
    for (name <- args) println(
      colors.get(name) match {
        case Some(code) =>
          name + " has code: " + code
        case None =>
          "Unknown color: " + name
      }
    )
  }
}
```

Anexa B

Alte informații relevante (demonstrații etc.)

Anexa C

Lucrări publicate (dacă există)